

Overview of Rectifier neural networks

Subject: Rectifier neural networks

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

1.

$$\text{LSE}_0 + \sigma(x_1, \dots, x_n) := \text{LSE}_0(0, x_1, \dots, x_n) = \ln(1 + e^{x_1} + \dots + e^{x_n})$$
$$\text{LSE}_0(x_1, \dots, x_n) := \ln(1 + e^{x_1} + \dots + e^{x_n})$$

2.

History The ReLU was first used by Alston Householder in 1941 as a mathematical abstraction of biological neural networks. Kunihiro Fukushima in 1969 used ReLU in the context of visual feature extraction in hierarchical neural networks. In 1998, Gregory Woodbury demonstrated that the rectified linear function could account for a broad range of emergent properties in the visual cortex. His work showed that a single unified model could drive the joint development of refined retinotopic maps, ocular dominance columns, and orientation selectivity. By utilizing the rectifier's "cutoff" property, Woodbury achieved a close quantitative fit to biological data, matching the spatial periodicities and topographic refinement patterns observed in macaque and cat cortical maps. Furthermore, he extended this framework to adult plasticity, accurately replicating the spatial and temporal dynamics of lesion-induced cortical reorganization. This research established that the rectified linear response was a necessary mechanism for the stable self-organisation and maintenance of complex, multi-feature neural maps. In 2000, Hahnloser et al. argued that ReLU approximates the biological relationship between neural firing rates and input current, in addition to enabling recurrent neural network dynamics to stabilise under weaker criteria. Prior to 2010, most activation functions used were the logistic sigmoid (which is inspired by probability theory; see logistic regression) and its more numerically efficient counterpart, the hyperbolic tangent. Around 2010, the use of ReLU became common again. Jarrett et al. (2009) noted that rectification by either absolute or ReLU (which they called "positive part") was critical for object recognition in convolutional neural networks (CNNs), specifically because it allows average pooling without neighboring filter outputs cancelling each other out. They hypothesized that the use of sigmoid or tanh was responsible for poor performance in previous CNNs. Nair and Hinton (2010) made a theoretical argument that the softplus activation function should be used, in that the softplus function numerically approximates the sum of an exponential number of linear models that share parameters. They then proposed ReLU as a good approximation to it. Specifically, they began by considering a single binary neuron in a Boltzmann machine that takes x as input, and produces 1 as output with probability $\sigma(x) = \frac{1}{1 + e^{-x}}$. They then considered extending its range of output by making infinitely many copies of it $X_1, X_2, X_3, \dots$, that all take the same input, offset by an amount $0.5, 1.5, 2.5, \dots$, then their outputs are added together as $\sum_{i=1}^{\infty} X_i$. They then demonstrated that $\sum_{i=1}^{\infty} X_i$ is approximately equal to $N(\log(1 +$

$e^x), \sigma(x))$ $\{\displaystyle \{\mathcal{N}\}(\log(1+e^{\{x\}}), \sigma(x))\}$, which is also approximately equal to ReLU $\blacksquare (N(x, \sigma(x)))$ $\{\displaystyle \operatorname{ReLU}(\{\mathcal{N}\}(x, \sigma(x)))\}$, where N $\{\displaystyle \{\mathcal{N}\}\}$ stands for the gaussian distribution. They also argued for another reason for using ReLU: that it allows "intensity equivariance" in image recognition. That is, multiplying input image by a constant k $\{\displaystyle k\}$ multiplies the output also. In contrast, this is false for other activation functions like sigmoid or tanh. They found that ReLU activation allowed good empirical performance in restricted Boltzmann machines. Glorot et al (2011) argued that ReLU has the following advantages over sigmoid or tanh:

3.

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}, f'(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases} \quad f(x) = \begin{cases} 1 & x > 0 \\ \alpha & x \leq 0 \end{cases}$$

4.

In these formulas, a $\{\displaystyle a\}$, b $\{\displaystyle b\}$ and x_c $\{\displaystyle x_{\{c\}}\}$ are hyperparameter values which could be set as default constraints $a = 1$ $\{\displaystyle a=1\}$, $b = 2$ $\{\displaystyle b=2\}$ and $x_c = 1.25643$ $\{\displaystyle x_{\{c\}}=1.25643\}$, as done in the original work.

5.

In these formulas, α $\{\displaystyle \alpha\}$ is a hyperparameter to be tuned with the constraint $\alpha \geq 0$ $\{\displaystyle \alpha \geq 0\}$. Given the same interpretation of α $\{\displaystyle \alpha\}$, ELU can be viewed as a smoothed version of a shifted ReLU (SReLU), which has the form $f(x) = \max(-\alpha, x)$ $\{\displaystyle f(x) = \max(-\alpha, x)\}$.

6.

Piecewise-linear variants Leaky ReLU (2014) allows a small, positive gradient when the unit is inactive, helping to mitigate the vanishing gradient problem. This gradient is defined by a parameter α $\{\displaystyle \alpha\}$, typically set to 0.01–0.3.

7.

$$\operatorname{ReLU} \blacksquare (x) = x + \max(0, x) = x + |x|/2 = \begin{cases} x & \text{if } x > 0 \\ 0 & x \leq 0 \end{cases} \quad \operatorname{ReLU} \blacksquare (x) = x + \max(0, x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0 \\ 0 & x \leq 0 \end{cases}$$

8.

and its gradient is the softmax; the softmax with the first argument set to zero is the multivariable generalization of the logistic function. Both LogSumExp and softmax are used in machine learning.

9.

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1)x & x \leq 0 \end{cases}, f'(x) = \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases} \quad f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1)x & x \leq 0 \end{cases}, f'(x) = \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases}$$

10.

DELU Extended Exponential Linear Unit (DELU, 2023) is an activation function which is smoother within the neighborhood of zero and sharper for bigger values, allowing better allocation of neurons in the learning process for higher performance. Thanks to its unique design, it has been shown that DELU may obtain higher classification accuracy than ReLU and ELU.